

Demo Application 方案：Python Web App 双 CVE 依赖攻击链

0. Demo 定位说明

本项目是一个**受控靶场（Lab）Demo 应用**，用于演示：

- 开源依赖中的已知 CVE 如何被攻击者串联利用；
- AI 安全测试工具（如 Strix）如何发现和验证漏洞链；
- XDR 如何检测漏洞利用后的主机侧异常行为。

该应用并非生产系统设计，也不代表真实客户环境中的推荐架构。应用中的漏洞、配置文件、token、marker 文件以及 controlled beacon 均为演示目的而设计。

本 Demo 的目标是：

1. 构建一个业务上合理的 Python Web 应用场景
2. 在应用中引入两个存在已知 CVE 风险的 Python library
3. 展示攻击者如何从外部 Web 暴露面进入
4. 展示两个依赖漏洞如何被串联成完整攻击链
5. 让 Strix 等 AI 安全测试工具能够发现并验证该漏洞链
6. 让 XDR 检测到漏洞利用后的异常行为链

本 Demo 明确不包含：

真实生产凭据
真实云账号密钥
真实数据库密码
真实反弹 shell
真实可交互 C2
可泛化攻击工具

为了保证演示安全性，攻击结果应限制为：

读取 demo 专用配置文件
读取 demo 专用 profile-upload-token
创建 demo marker 文件
发起 controlled beacon
生成 XDR 可见的异常行为链

因此，本项目应被定位为：

一个用于 AI 安全测试与 XDR 检测演示的受控靶场应用，而不是可部署到生产环境的真实业务系统。

1. 最终方向

本 Demo 采用一个简化、统一、易于理解的设计：

一个 Python Web App，包含两个存在已知 CVE 风险的 Python library。

两个 library 分别承担不同攻击阶段：

aiohttp CVE

用于 Web 入口暴露、端口扫描 / HTTP 指纹识别、敏感配置读取

PyYAML CVE

用于业务配置文件解析阶段，触发受控执行效果

最终攻击链：

```
Port Scan / HTTP Fingerprinting
  ↓
发现 Python aiohttp Web 服务
  ↓
利用 aiohttp 目录穿越 / 任意文件读取风险
  ↓
读取本地 app_config.yaml 和 profile-upload-token
  ↓
获得 diagnostic-profile.yaml 上传能力
  ↓
上传特制 diagnostic-profile.yaml
  ↓
应用使用 vulnerable PyYAML 解析 YAML
  ↓
触发受控执行效果
  ↓
创建 demo marker / controlled beacon
  ↓
XDR 检测完整异常行为链
```

这个方案的核心价值是：

通过两个真实 Python 依赖库 CVE，把“外部入口发现 → 敏感配置读取 → 业务功能滥用 → 受控执行 → XDR 检测”串成一条完整、可解释、可重复的攻击链。

2. 业务场景

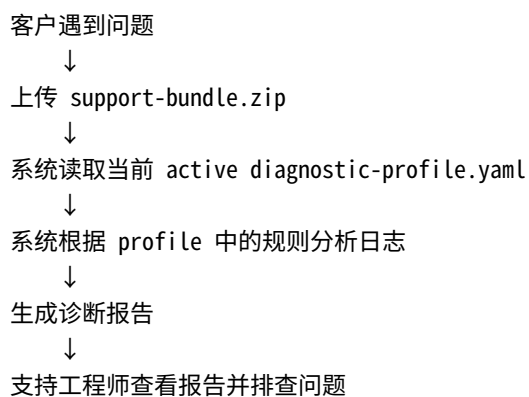
Demo Application 仍然采用一个合理的业务场景：

Customer Support Portal 客户支持门户

业务功能包括：

1. 客户上传 support-bundle.zip 日志包
2. 系统根据诊断规则分析日志
3. 系统生成 diagnostic report
4. App Owner / Support Admin 可以上传 diagnostic-profile.yaml

业务流程：



这个场景合理，因为真实支持系统经常需要：

- 处理客户上传的日志包；
- 支持不同产品线、不同日志格式；
- 通过配置文件定义日志匹配规则；
- 不改代码即可更新诊断逻辑。

3. diagnostic-profile.yaml 的用途

diagnostic-profile.yaml 是由 App Owner、Support Admin 或内部支持团队负责人 上传和维护的诊断规则配置文件。

普通客户不上传这个文件。普通客户只上传 support-bundle.zip。

角色分工：

角色	能做什么
普通客户	上传 support-bundle.zip，查看诊断报告
支持工程师	查看客户日志与诊断报告
App Owner / Support Admin	上传或更新 diagnostic-profile.yaml

角色

能做什么

系统后台

根据 profile 分析日志、生成报告

`diagnostic-profile.yaml` 的作用是告诉系统：

检查哪些日志文件
匹配哪些错误关键字
如何定义 severity
报告里包含哪些章节
给出哪些 recommended actions

示例：

```
profile_name: "Linux Agent Basic Diagnostic"
version: "1.0"

log_sources:
- name: "agent log"
  path: "agent.log"
  type: "text"

- name: "system log"
  path: "system.log"
  type: "text"

rules:
- id: "AGENT-001"
  description: "Agent failed to connect to management server"
  severity: "high"
  match:
    file: "agent.log"
    contains: "connection refused"

- id: "AGENT-002"
  description: "Authentication token expired"
  severity: "medium"
  match:
    file: "agent.log"
    contains: "token expired"

- id: "SYS-001"
  description: "Disk space is low"
  severity: "medium"
  match:
    file: "system.log"
    contains: "No space left on device"

report:
```

```
title: "Linux Agent Diagnostic Report"
include_sections:
  - "summary"
  - "matched_rules"
  - "recommended_actions"
```

正常情况下，这只是一个规则配置文件，不应该带来执行风险。

漏洞来自于：

应用使用了存在 CVE 风险的 PyYAML 解析方式处理这个 YAML 文件。

4. 第一个 CVE：aiohttp 入口漏洞

4.1 角色定位

第一个 CVE 负责攻击链的入口阶段：

```
外部扫描
↓
发现 aiohttp Web 服务
↓
识别存在目录穿越 / 任意文件读取风险
↓
读取本地配置和 demo token
```

这个阶段的目标不是直接获得 shell，而是读取本来不该被外部访问的本地文件。

4.2 为什么它可以被扫描发现

这里的“端口扫描发现”应准确表述为：

端口扫描 + HTTP 指纹识别 + 漏洞探测

纯端口扫描只能发现：

目标开放了 Web 端口

进一步通过 HTTP 指纹、响应头、静态资源路径探测、版本识别等方式，扫描器可以判断目标疑似使用 vulnerable aiohttp，并进一步验证是否存在目录穿越风险。

对外更严谨的说法是：

第一个 CVE 可以通过外部暴露面扫描发现疑似风险，再通过 HTTP 层漏洞探测进行验证。

4.3 Demo 中读取什么文件

为了安全、可控、易解释，建议只读取 demo 专用文件，而不是读取真实系统密钥。

推荐设计两个文件：

```
/opt/support-portal/config/app_config.yaml
/opt/support-portal/secrets/profile-upload-token
```

文件一： `app_config.yaml`

作用：告诉攻击者内部 profile 上传接口在哪里，以及 token 文件路径在哪里。

示例：

```
app:
  name: "Customer Support Portal"
  mode: "demo"

public_routes:
  support_bundle_upload: "/support/upload"
  report_download: "/support/report/{ticket_id}"

internal_routes:
  diagnostic_profile_upload: "/internal/app-owner/profile/upload"

secrets:
  profile_upload_token_file: "/opt/support-portal/secrets/profile-upload-token"
```

攻击者通过第一个 CVE 读取该配置后，知道存在内部接口：

```
/internal/app-owner/profile/upload
```

文件二： `profile-upload-token`

作用：保护 `diagnostic-profile.yaml` 上传接口。

示例：

```
demo-profile-admin-token-2026
```

这个 token 不是 root 密码、云账号密钥或真实生产凭据。它只是 demo 应用内部用于保护 profile 上传接口的一个 bearer token。

正常情况下：

只有 App Owner / 管理后台服务知道这个 token
外部客户用户不应该知道

但第一个 CVE 让攻击者可以读取这个文件，从而获得上传 `diagnostic-profile.yaml` 的能力。

5. 第二个 CVE：PyYAML 解析漏洞

5.1 角色定位

第二个 CVE 负责攻击链的执行阶段：

攻击者获得 profile 上传能力
↓
上传特制 `diagnostic-profile.yaml`
↓
后端使用 vulnerable PyYAML loader 解析 YAML
↓
触发受控执行效果

这个阶段展示的是：

一个本来只是配置文件的 YAML，由于被不安全解析，变成了执行入口。

5.2 为什么这个设计合理

`diagnostic-profile.yaml` 是内部支持系统的正常配置文件。

它的正常用途是：

定义日志源
定义错误匹配规则
定义 severity
定义报告章节
定义建议动作

但是如果应用用不安全的 PyYAML loader 处理它，就可能出现反序列化 / 代码执行风险。

在 Demo 中，第二阶段不要设计成真实攻击工具，而是设计成受控结果，例如：

创建 demo marker 文件
或
向 `demo-c2.local` 发起固定 controlled beacon

或
触发一条 XDR 可见的异常 Python 行为

这样既能展示风险，又不会把 Demo 变成真正的远控工具。

6. 两个 CVE 的串联逻辑

完整链路如下：

1. 攻击者扫描到 Python Web 服务
- ↓
2. 通过 HTTP 指纹识别发现目标疑似 aiohttp
- ↓
3. 进一步探测发现存在 aiohttp 目录穿越 / 文件读取风险
- ↓
4. 读取 /opt/support-portal/config/app_config.yaml
- ↓
5. 从配置中发现 profile 上传接口路径
- ↓
6. 读取 /opt/support-portal/secrets/profile-upload-token
- ↓
7. 使用 token 访问 /internal/app-owner/profile/upload
- ↓
8. 上传特制 diagnostic-profile.yaml
- ↓
9. 后端使用 vulnerable PyYAML 解析该 YAML
- ↓
10. 触发受控执行效果
- ↓
11. 创建 demo marker 或发起 controlled beacon
- ↓
12. XDR 检测到完整异常行为链

简化表达：

aiohttp CVE：读配置、读 token
↓
PyYAML CVE：上传 YAML、触发执行
↓
XDR：检测异常文件读取、异常解析行为、异常外连或 marker 创建

7. 攻击前后对比

攻击前

外部客户只能访问普通 support portal
外部客户只能上传 support-bundle.zip
外部客户不知道 profile 上传接口
外部客户没有 profile-upload-token
外部客户不能上传 diagnostic-profile.yaml

攻击后

攻击者通过 aiohttp CVE 读取 app_config.yaml
攻击者发现内部 profile 上传接口
攻击者通过 aiohttp CVE 读取 profile-upload-token
攻击者上传特制 diagnostic-profile.yaml
PyYAML CVE 被触发
系统产生受控执行效果
XDR 发现攻击链

8. XDR 检测点

XDR 可以观察到的行为包括：

1. 外部来源对 Web 服务进行扫描和异常路径访问
2. Python aiohttp 进程读取异常本地配置文件
3. Python aiohttp 进程访问 secret 文件
4. profile 上传接口被异常调用
5. diagnostic-profile.yaml 被更新
6. Python 进程使用 PyYAML 解析异常 YAML
7. Python 进程创建 demo marker 文件
8. Python 进程发起 controlled beacon 外连
9. 多个事件被关联为一条攻击链

对客户可以解释为：

XDR 不只是看到一个孤立告警，而是看到从 Web 暴露面探测、敏感文件读取、内部接口滥用、YAML 解析异常，到外连行为的一整条攻击链。

9. Strix / AI 渗透测试工具的定位

这个 Demo 适合 Strix 这类 AI 安全测试工具做：

源码分析 + 依赖分析 + 动态验证

更推荐的输入方式是：

源码仓库
requirements.txt
运行中的 Web URL
测试账号
业务说明 README

Strix 可以分析：

1. requirements.txt 中存在 vulnerable aiohttp 版本
2. Web App 使用 aiohttp static route
3. static route 配置可能触发目录穿越风险
4. requirements.txt 中存在 vulnerable PyYAML 版本
5. App 使用不安全 YAML loader 解析 diagnostic-profile.yaml
6. 第一个 CVE 可以读取 profile 上传 token
7. 第二个 CVE 可以通过 profile 上传功能触发

更准确的 Demo 定位是：

Strix 负责发现并验证应用依赖漏洞链；XDR 负责检测漏洞被利用后的主机行为链。

10. 是否保留提权叙事

这个简化版中，不建议再强行强调 OS 层面的 root 提权。

原因是：

整个 Demo 只有一个 Python Web App
两个 CVE 都发生在应用依赖库中
第二个 CVE 触发的是应用进程上下文中的受控执行

所以更准确的攻击结果是：

敏感配置读取
内部接口滥用
应用进程内受控执行
可疑外连 / marker 创建
XDR 检测攻击链

而不是：

低权限 Web 用户 → root 权限

如果未来要加入“提权”，可以再引入：

高权限 diagnostic-worker
或
本地系统组件 LPE CVE

但在当前版本中，为了保持简单，建议先不加入 OS 提权。

11. Demo 中应避免的设计

不建议在 Demo 中设计：

真实反弹 shell
真实可交互 C2
真实云账号密钥
真实数据库密码
/root/.ssh/id_rsa
可泛化利用脚本

建议只使用：

demo 专用 token
demo 专用配置文件
demo marker 文件
controlled beacon
固定目的地址
固定请求内容
不接受任意远程命令

这样 Demo 更安全、更可控，也更适合客户现场演示。

12. 对销售和工程师的解释方式

销售版本

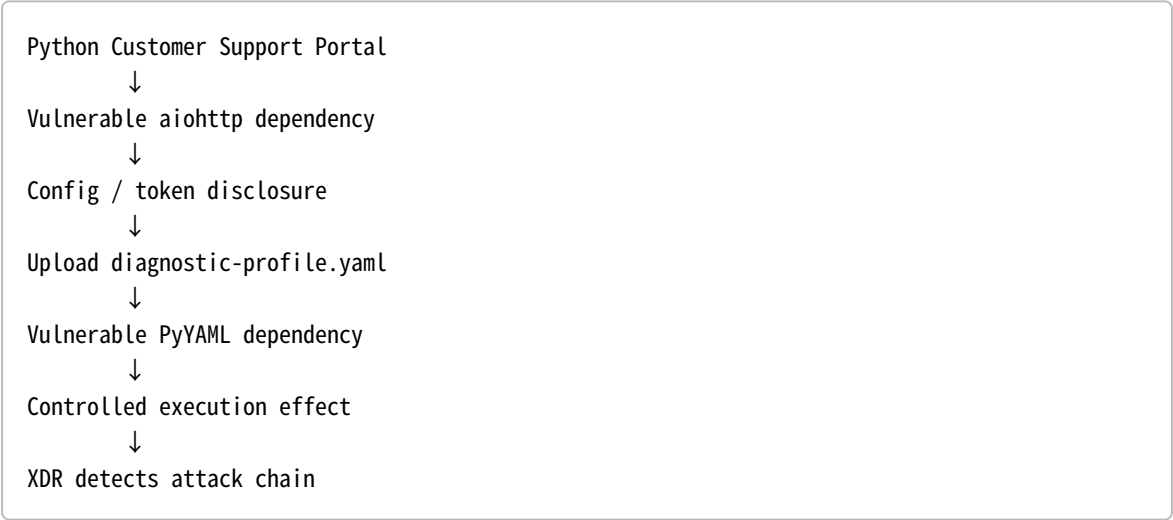
这个 Demo 展示的是：一个 Python 客户支持门户由于使用了两个存在 CVE 风险的开源依赖，形成了一条完整攻击链。攻击者先通过 Web 入口层的 aiohttp 漏洞读取内部配置和上传 token，再利用 PyYAML 的不安全解析问题触发受控执行行为。XDR 可以把扫描、文件读取、内部接口滥用、异常执行和外连行为关联成完整攻击链。

工程师版本

该应用使用 aiohttp 提供 Web 服务，并暴露静态资源路由。在特定配置下，aiohttp 的目录穿越漏洞可导致外部攻击者读取本地配置文件。攻击者读取 `app_config.yaml` 后发现 `diagnostic-profile.yaml` 上传接口，并进一步读取 `profile-upload-token`。随后攻击者调用内部 profile 上传接口，上传特制 YAML。应用使用有风险的 PyYAML loader 解析该 YAML，触发受控执行效果。XDR 可在主机侧观察到异常文件读取、异常 YAML 处理、marker 创建或 controlled beacon 外连。

13. 最终方案总结

最终 Demo 方案：



该方案满足以下要求：

要求	是否满足
一个 Python Web App	满足
两个有 CVE 问题的 Python library	满足
其中一个 CVE 可由外部扫描发现疑似风险	满足，aiohttp 入口可通过端口扫描 + HTTP 指纹 + 漏洞探测发现
两个 CVE 可以形成攻击链	满足，aiohttp 读配置 / token，PyYAML 触发执行
业务场景合理	满足，客户支持门户 + 诊断规则配置
普通工程师能理解	基本满足，读配置 → 拿 token → 上传 YAML → 触发执行
销售能讲清楚	基本满足，需要简化成“两个开源依赖漏洞串成攻击链”
Strix 有机会发现	满足，适合源码 + 依赖 + 动态验证
XDR 有检测点	满足，有扫描、文件读取、接口滥用、异常执行、外连行为
Demo 可重复	满足，使用固定配置、固定 token、固定 marker / beacon
不做真实远控	满足，采用 controlled beacon 或 marker

再次强调：

本项目是一个用于安全演示、漏洞验证和检测能力展示的靶场 Demo 应用。所有漏洞利用结果均为受控行为，目的是帮助展示 AI 安全测试工具与 XDR 的能力，而非构建真实攻击工具或生产环境系统。